TEMPLE DONATION AND MANAGEMENT SYSTEM

Architecture Design Analysis and Report

Student Name	Y. HEMA SAI LAHARI
Register Number	192311446
Course	Software Engineering
Assessment	CO2 – Architecture Design Assignment

1. System Requirement Analysis

The Temple Donation and Management System aims to provide a secure, reliable, efficient, and user-friendly platform for managing temple donations and administrative activities.

1.1 System Objectives

• Automate donation management.

• Provide secure online donation facilities.

• Manage devotee information.

• Maintain donation and payment records.

• Generate digital receipts.

• Manage temple events.

• Generate reports.

• Protect sensitive information.

• Support increasing users and transactions.

• Reduce manual effort and improve efficiency.

1.2 Functional Requirements

• User Registration and Login

• Devotee Profile Management

• Donation Management

• Online Payment Processing

• Digital Receipt Generation

• Donation History

• Event Management

• Donor Management

• Report Generation

• Notifications

• Administrator Management

• Search and Data Retrieval

1.3 Non-Functional Requirements

• Performance: Fast processing and retrieval.

• Scalability: Support increasing devotees and donations.

• Security: Authentication, authorization, secure payment communication, and data protection.

• Reliability: Accurate and safe donation records.

• Availability: Accessible whenever required.

• Maintainability: Easy modification and testing.

• Usability: Simple interface.

• Portability: Web and mobile access where required.

2. Selected Software Architecture

Selected Architecture: Layered Architecture

The proposed system is divided into Presentation, Business Logic, Data Access, and Database layers. External services such as payment gateways and notification services communicate through secure APIs.

2.1 Architecture Structure

```
USERS: Devotee | Administrator | Temple Management
                          ↓ HTTPS
PRESENTATION LAYER: Login | Dashboard | Donation | Events | Reports
                          ↓
BUSINESS LOGIC LAYER: Authentication | Donation Processing | Validation | Events | Reports
                          ↓
DATA ACCESS LAYER: Repository / DAO | CRUD | Queries
                          ↓
DATABASE: Users | Donations | Payments | Receipts | Events | Reports | Logs
                          ↕
EXTERNAL SERVICES: Payment Gateway | Email/SMS API
```

2.2 Why Layered Architecture is Suitable

• Clear separation of responsibilities.

• Presentation handles interaction.

• Business layer handles rules and processing.

• Data access manages database operations.

• Database stores application data.

• Easier development, testing, debugging, maintenance, and upgrades.

• Users do not directly access the database.

2.3 Advantages

• Simple and easy to understand

• Clear separation of responsibilities

• Easy maintenance

• Easy testing

• Better security

• Code reuse

• External service integration

• Future enhancements

2.4 Limitations

• Additional processing overhead

• Some changes may affect multiple layers

• Large monolithic systems may be difficult to scale independently

3. Detailed Architecture Diagram

```
                +-----------------------------+
                | USERS                       |
```

4. Components and Their Interactions

4.1 Presentation Layer

Provides screens for login, registration, dashboard, donation, payment, receipt, donation history, events, and reports. It accepts input, performs basic validation, sends requests, and displays results.

4.2 Business Logic Layer

Authenticates users, applies roles and permissions, validates donations, processes requests, manages events, generates receipts and reports, and coordinates external services.

4.3 Data Access Layer

Inserts, retrieves, updates, and deletes records; executes database queries; and hides database implementation details.

4.4 Database Layer

Stores user information, devotee profiles, donations, payment transactions, receipts, events, announcements, reports, and system logs.

4.5 External APIs/Services

Payment Gateway processes online donations. Email/SMS service sends confirmations, receipts, and event notifications. Communication occurs through secure APIs.

5. Overall System Workflow

```
USER → USER INTERFACE → BUSINESS LOGIC → DATA ACCESS → DATABASE
                          ↓
                  PAYMENT GATEWAY
                          ↓
                 TRANSACTION SUCCESS
                          ↓
                  STORE DONATION
                          ↓
                  GENERATE RECEIPT
                          ↓
                 EMAIL / SMS SERVICE
                          ↓
                         USER
```

6. Donation Processing Flowchart

```
START → Login → Donation Page → Enter Amount → Validate
                                                ↓
                                             Valid?
                                           /        \
                                         No          Yes
                                         ↓            ↓
                                  Error Message   Payment Gateway
                                                      ↓
                                                Payment Successful?
                                                  /            \
                                                No              Yes
                                                ↓               ↓
                                          Payment Failed   Save Transaction
                                                                ↓
                                                          Generate Receipt
                                                                ↓
                                                         Send Notification
                                                                ↓
                                                               END
```

7. Database Interaction Flow

```
USER
  ↓
PRESENTATION LAYER
  ↓
BUSINESS LOGIC LAYER
  ↓
DATA ACCESS LAYER
  ↓
DATABASE
  ■■■ Users
  ■■■ Donations
  ■■■ Payments
  ■■■ Receipts
  ■■■ Events
```

```
  ■■■ Reports / Logs
  ↓
RESULT → BUSINESS LOGIC → PRESENTATION → USER
```

8. Quality Attribute Analysis

Quality Attribute	Architecture Support
Scalability	Additional resources, indexing, caching, and optimized components support growth.
Security	Authentication, role-based authorization, HTTPS, hashing, validation, restricted database access, and secure APIs.
Performance	Efficient queries, indexing, caching, optimized logic, and reduced unnecessary API calls.
Reliability	Validation, error handling, database transactions, backups, recovery, logging, and monitoring.
Maintainability	Separate responsibilities allow individual layers to be modified with minimal impact.
Availability	Backups, monitoring, recovery mechanisms, health checks, and redundant deployment where required.

9. Architectural Justification

Layered Architecture was selected because the system requires clear separation between user interaction, donation processing, data management, and database operations. It supports modular development, easier testing, security enforcement, database management, future features, and external API integration.

10. Architectural Trade-Offs

Decision	Advantage	Trade-Off
Layered Architecture	Simple and maintainable	Some processing overhead
Central Database	Easy data management	Can become a bottleneck
Payment API	Online payment integration	Depends on external service
Authentication Layer	Improved security	Additional processing complexity
Data Access Layer	Easier database maintenance	Additional abstraction

11. Future Enhancements

• Dedicated Android/iOS application

• Multiple temple branch support

• Multilingual interface

• QR-based donation receipts

• Advanced financial analytics

• Automated SMS and email notifications

• Cloud deployment

• Online event registration

• Recurring donations

• Digital donation certificates

• Advanced dashboards

• Additional payment services

• Migration of selected services to microservices if system growth requires it

12. Long-Term Maintainability

• UI changes can be made in the Presentation Layer.

• Donation and business-rule changes can be made in the Business Logic Layer.

• Database-related changes can be handled in the Data Access Layer.

• Database structure can be modified independently where possible.

• Separation reduces the impact of changes and supports long-term development.

13. Conclusion

The Temple Donation and Management System uses Layered Architecture to provide a structured and maintainable software solution. The architecture separates presentation, business logic, data access, and database responsibilities. It addresses scalability, security, performance, reliability, availability, and maintainability while supporting payment gateway integration, notifications, future mobile applications, advanced reporting, and cloud deployment.